Supplementary Material for "Evaluating LLM-based Regression Test Generation"

ANONYMOUS AUTHOR(S)

This document provides supplementary material for the paper *Evaluating LLM-based Regression Test Generation*. It includes:

(1) Commit message enhancement experiment setup and prompts
(2) Analysis of WAFLGo failures on certain subject programs and commits
(3) The complete results of ClevFuzz (extending Table 4 from the main paper)
(4) Experimental details and results of the *unguided*-Cleverest experiment
(5) Code coverage results for 50 recent non-bug commits
(6) Commit message size statistics

The raw data and scripts used to generate these results are available at https://anonymous.4open.science/r/cleverest-7767/.

ACM Reference Format:
Anonymous Author(s). 2026. Supplementary Material for "Evaluating LLM-based Regression Test Generation". 1, 1 (February 2026), 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

1 Commit Message Enhancement Configuration

We use the following prompt to automatically enhance commit message based on original commit info and issue bug report:

> You are an expert software developer and security tester.
> Please browse the url of bug issue report and bug fixing commit below.
> The original commit message may not accurately reflect the nature of the bug and intent of the bugfix.
> Please enhance the commit message so it describes condition to reach/trigger the bug concisely and informatively.
> The goal is that seeing the enhanced message it is enough to construct high-quality regression test.
> The enhanced message should keep the original message style/structure, preferably a single complete sentence. If original message contains body, keep it intact, otherwise do NOT add body.
> Do NOT directly include original/example test case in the message. Prefer natural language over code/implementation detail unless it is necessary. Only reply with the enhanced commit message, nothing else.
> Issue Report: URL of ISSUE
> Bugfix commit:
> COMMIT INFO WITH MSG AND DIFF

We used `gemini-2.5-flash` because it supports accepting URL as context so we do not need to manually fetch and paste the issue report content into the prompt. We expect developers can use any LLMs or AI-powered IDE, or even manually enhance the commit message to make it more informative for regression test generation.

2 WAFLGo Failure Analysis

In our evaluation, WAFLGo encountered three types of failures that prevented it from running on several subject programs and commits:

(1) **Static analysis phase OOM**: On Z3 and PHP, WAFLGo's static analysis phase (based on SVF) consistently ran out of memory, exceeding 251 GB of RAM.
(2) **Build instrumentation failure**: One commit in JerryScript (`b7e3bae`) failed to build with WAFLGo's instrumentation with clang crashing.
(3) **Fuzzer runtime abort**: WAFLGo experienced immediate runtime aborts for one commit in JerryScript (`de51531`) and four commits in MicroPython (`e702046`, `f397a3e`, `c0252d7`, and `4b013ec`). We found that other researchers have reported similar runtime error on WAFLGo's GitHub repository (Issue #2[1]), but the authors have not yet responded to these reports.

We attempted our best to set up WAFLGo for all cases but could not resolve these technical limitations. We attached scripts to build WAFLGo images for all subjects in our dataset and actual failure logs in the replication package.

[1] https://github.com/NESA-Lab/WAFLGo/issues/2

3 Full ClevFuzz Results

Table 1 shows the full result of Cleverest and LLM-seeded fuzzer ClevFuzz on 72 commits from 8 subjects, including all 16 commits of Z3, PHP, 2 commits in JerryScript, and 4 commits in MicroPython where WAFLGo cannot be used for comparison due to building failure.

In the bug-finding scenario, WAFLGo successfully triggered 10 out of 36 bugs, while ClevFuzz triggered 22 out of 36. In the bug-reproduction scenario, WAFLGo reproduced 8 out of 36 bugs, while ClevFuzz reproduced 20 out of 36.

Beyond the intended bugs in the benchmark, ClevFuzz unexpectedly generated a bug-triggering test case while fuzzing #6659, leading to the discovery of a previously unknown heap-use-after-free bug in Z3. This bug could crash the latest stable version at the time of our experiment. **We immediately reported the issue to the developers, who confirmed and fixed it in the next release.**

Table 1. Full results for bug finding and bug reproduction effectiveness of CLEVEREST and CLEVEREST + fuzzing (CLEVFUZZ) on all 36 bugs. For every configuration, we show the average effectiveness score on a slider (✘ : Not reached; ➢ : Reached; ✐ : Output-changing; ✓ : Bug-revealing), number of bug-triggering repetitions (Bug) and time-to-exposure (*T.O.* means timeout after 24 hours).

	Subject (# seeds)	Issue	WAFLGo Init	WAFLGo Bug	WAFLGo T (h:m:s)	✘ ➢ ✐ ✓	CLEVEREST Reach	CLEVEREST Change	CLEVEREST Bug	CLEVEREST T (h:m:s)	CLEVFUZZ Bug	CLEVFUZZ T (h:m:s)	CLEVFUZZ Bug_{all}
Bug-finding	Mujs (19)	#65	✘	10/10	04:43:56		10/10	10/10	10/10	00:00:07	-/-	-	10/10
		#141	✘	0/10	T.O.		10/10	10/10	0/10	00:00:45	10/10	03:25:05	10/10
		#145	✘	10/10	00:17:21		10/10	8/10	8/10	00:00:37	2/2	00:00:00	10/10
		#166	✓	10/10	T.O.		10/10	1/10	0/10	00:00:57	0/10	T.O.	0/10
	Libxml2 (14)	#535	✓	10/10	T.O.		10/10	10/10	10/10	00:00:05	-/-	-	10/10
		#550	✐	0/10	T.O.		10/10	2/10	0/10	00:00:27	0/10	T.O.	0/10
		#553	✓	10/10	T.O.		10/10	10/10	10/10	00:00:06	-/-	-	10/10
		#720	✐	0/10	T.O.		10/10	0/10	0/10	00:00:53	4/10	18:08:01	4/10
		#841	✓	10/10	T.O.		10/10	10/10	10/10	00:00:05	-/-	-	10/10
	Poppler (100)	#1282	➢	6/10	00:06:23		8/10	3/10	0/10	00:03:16	8/8	00:00:10	8/10
		#1289	➢	0/10	T.O.		4/10	0/10	0/10	00:02:45	3/4	06:00:04	3/10
		#1303	✘	0/10	T.O.		0/10	0/10	0/10	00:03:38	-/-	-	0/10
		#1305	✘	0/10	T.O.		0/10	0/10	0/10	00:04:27	-/-	-	0/10
		#1381	➢	0/10	T.O.		0/10	0/10	0/10	00:03:22	-/-	-	0/10
	JerryScript (19)	#5013	✘	-	-		10/10	0/10	0/10	00:01:46	1/10	21:36:10	1/10
		#5117	➢	4/10	00:13:02		10/10	10/10	0/10	00:01:11	10/10	00:01:41	10/10
		#5138	➢	0/10	T.O.		10/10	10/10	2/10	00:01:18	8/8	00:00:37	10/10
		#5153	➢	0/10	T.O.		10/10	10/10	0/10	00:01:35	10/10	00:00:59	10/10
	Z3	#6659	-	-	-		0/10	0/10	0/10	00:02:16	-/-	-	0/10
		#6914	-	-	-		2/10	0/10	0/10	00:03:08	0/2	T.O.	0/10
		#7246	-	-	-		10/10	0/10	0/10	00:01:15	0/10	T.O.	0/10
		#7252	-	-	-		8/10	0/10	0/10	00:01:40	0/8	T.O.	0/10
	PHP	#16777	-	-	-		10/10	0/10	0/10	00:01:49	0/10	T.O.	0/10
		#16978	-	-	-		10/10	1/10	0/10	00:01:46	5/10	13:25:48	5/10
		#17442	-	-	-		10/10	10/10	0/10	00:01:43	0/10	T.O.	0/10
		#17463	-	-	-		6/10	0/10	0/10	00:02:02	0/6	T.O.	0/10
	JQ (38)	#2825	➢	0/10	T.O.		10/10	0/10	0/10	00:00:23	10/10	04:42:34	10/10
		#2976	➢	2/10	01:55:03		10/10	0/10	0/10	00:00:22	10/10	01:21:39	10/10
		#3262	✓	10/10	T.O.		10/10	10/10	10/10	00:00:09	-/-	-	10/10
		#49014	✘	0/10	T.O.		2/10	2/10	0/10	00:00:32	2/2	07:51:24	2/10
	MicroPython (13)	#13007	➢	0/10	T.O.		9/10	7/10	0/10	00:00:27	7/9	07:07:12	7/10
		#13041	✘	0/10	T.O.		0/10	0/10	0/10	00:00:31	-/-	-	0/10
		#17733	➢	0/10	T.O.		10/10	0/10	0/10	00:00:26	6/10	12:35:08	6/10
		#17815	➢	0/10	T.O.		10/10	9/10	0/10	00:00:26	4/10	17:54:34	4/10
		#17841	✘	-	-		10/10	10/10	0/10	00:00:21	0/10	T.O.	0/10
		#17847	✘	0/10	T.O.		10/10	10/10	0/10	00:00:40	2/10	20:33:41	2/10
	Aggregate			10/36	19:39:50		31/36	20/36	7/36	00:01:19		09:45:54	22/36
Bug-reproduction	Mujs (19)	#65	➢	0/10	T.O.		10/10	6/10	6/10	00:00:19	1/4	18:03:42	7/10
		#141	✘	5/10	00:12:47		10/10	10/10	1/10	00:00:31	9/9	01:43:32	10/10
		#145	✘	10/10	00:08:26		10/10	9/10	9/10	00:00:28	1/1	00:00:03	10/10
		#166	✓	10/10	T.O.		10/10	10/10	10/10	00:00:07	-/-	-	10/10
	Libxml2 (14)	#535	✓	10/10	T.O.		10/10	10/10	10/10	00:00:05	-/-	-	10/10
		#550	✘	0/10	T.O.		0/10	0/10	0/10	00:00:29	-/-	-	0/10
		#553	✓	10/10	T.O.		10/10	10/10	10/10	00:00:13	-/-	-	10/10
		#720	✐	0/10	T.O.		7/10	0/10	0/10	00:01:44	7/7	03:09:11	7/10
		#841	✓	10/10	T.O.		10/10	10/10	10/10	00:00:05	-/-	-	10/10
	Poppler (100)	#1282	➢	0/10	T.O.		9/10	0/10	0/10	00:02:03	1/9	22:09:30	1/10
		#1289	➢	0/10	T.O.		8/10	6/10	6/10	00:01:30	2/2	00:00:08	8/10
		#1303	✘	0/10	T.O.		1/10	1/10	0/10	00:02:12	0/1	T.O.	0/10
		#1305	➢	0/10	T.O.		0/10	0/10	0/10	00:02:27	-/-	-	0/10
		#1381	➢	0/10	T.O.		0/10	0/10	0/10	00:02:27	-/-	-	0/10
	JerryScript (19)	#5013	✘	0/10	T.O.		10/10	0/10	0/10	00:01:12	10/10	03:02:42	10/10
		#5117	➢	0/10	T.O.		10/10	5/10	5/10	00:00:50	2/5	20:54:43	7/10
		#5138	✘	0/10	T.O.		10/10	0/10	0/10	00:01:14	4/10	17:50:21	4/10
		#5153	✘	-	-		0/10	0/10	0/10	00:00:39	-/-	-	0/10
	Z3	#6659	-	-	-		0/10	0/10	0/10	00:01:14	-/-	-	0/10
		#6914	-	-	-		0/10	0/10	0/10	00:01:16	-/-	-	0/10
		#7246	-	-	-		0/10	0/10	0/10	00:01:08	-/-	-	0/10
		#7252	-	-	-		5/10	0/10	0/10	00:01:16	0/5	T.O.	0/10
	PHP	#16777	-	-	-		10/10	0/10	0/10	00:01:27	0/10	T.O.	0/10
		#16978	-	-	-		0/10	0/10	0/10	00:01:10	-/-	-	0/10
		#17442	-	-	-		0/10	0/10	0/10	00:01:18	-/-	-	0/10
		#17463	-	-	-		10/10	10/10	10/10	00:00:09	-/-	-	10/10
	JQ (38)	#2825	➢	0/10	T.O.		10/10	1/10	0/10	00:00:28	2/5	19:20:31	2/10
		#2976	➢	1/10	19:03:38		10/10	10/10	10/10	00:00:03	-/-	-	10/10
		#3262	✓	10/10	T.O.		10/10	10/10	10/10	00:00:08	-/-	-	10/10
		#49014	✘	0/10	T.O.		0/10	0/10	0/10	00:00:32	-/-	-	0/10
	MicroPython (13)	#13007	✐	-	-		1/10	1/10	0/10	00:00:26	0/1	T.O.	0/10
		#13041	✐	0/10	T.O.		10/10	10/10	0/10	00:00:21	10/10	02:58:33	10/10
		#17733	✐	0/10	T.O.		10/10	0/10	0/10	00:00:31	5/10	15:00:18	5/10
		#17815	✐	-	-		10/10	10/10	10/10	00:00:05	-/-	-	10/10
		#17841	✐	-	-		10/10	0/10	0/10	00:00:39	0/10	T.O.	0/10
		#17847	✐	0/10	T.O.		0/10	0/10	0/10	00:00:29	-/-	-	0/10
	Aggregate			8/36	21:48:32		25/36	17/36	13/36	00:00:52		06:47:54	20/36

4 Unguided-Cleverest

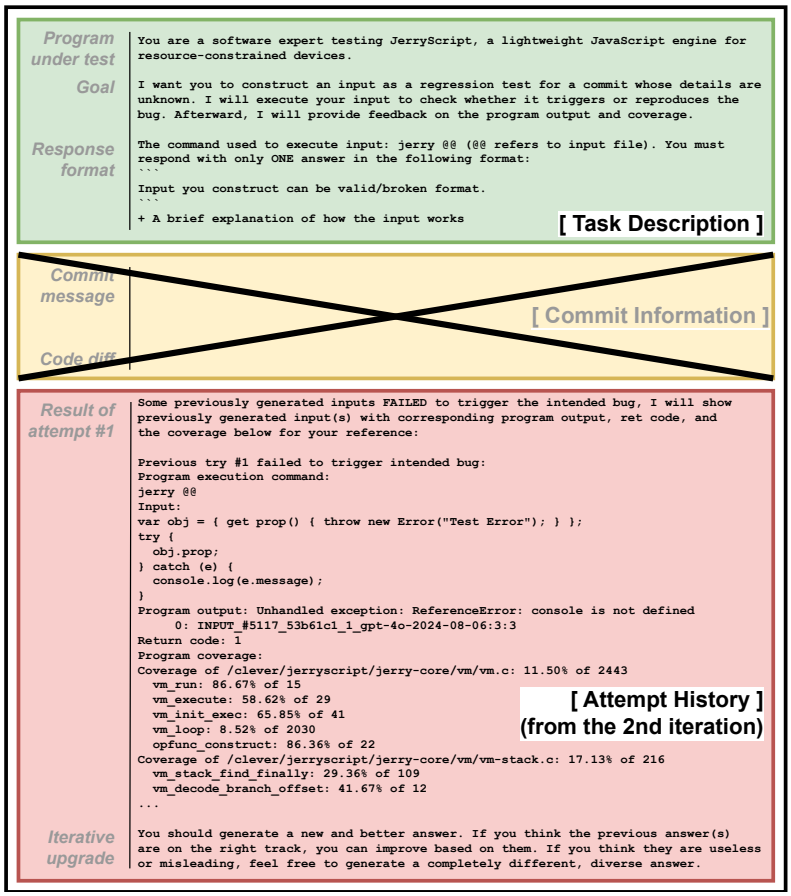

Fig. 1. The prompt generated by the prompt synthesizer for the unguided-Cleverest to generate the test input for JerryScript.

4.1 Purpose

The purpose of this experiment is to compare the effectiveness of Cleverest with other LLM-based test generation tools for regression testing. To facilitate a fair comparison, we implement an unguided version of Cleverest, which does not use commit information or execution feedback, aligning it with general LLM-based test generation tools such as Fuzz4All [1].

4.2 Why Not Fuzz4All?

Comparing Fuzz4All directly with Cleverest poses challenges due to fundamental differences in design choices and use cases, making a fair evaluation difficult. Fuzz4All uses OpenAI’s GPT-4 to distill user-provided documents into initial prompts and then employs a local LLM (StarCoder) to continuously generate and mutate inputs over 24 hours, without leveraging execution feedback. Maintaining Fuzz4All’s original settings would create an unfair advantage, as it

can generate significantly more inputs (~30k) compared to our method (only 5 per commit). However, it also has a disadvantage in relying on a less advanced LLM. Modifying their tool for a fair comparison is also impractical, as it would require addressing numerous confounding factors. The same issue applies to comparisons with other LLM-based test generation tools, as none of them specifically target regression test generation.

To highlight our key insight—leveraging commit information and execution feedback—we implement an unguided version of Cleverest as a proxy for general LLM-based test generation tools, enabling a more meaningful comparison.

4.3 Prompt of *unguided*-Cleverest

The *unguided*-Cleverest operates without access to commit information, assuming that only a single version of the program under test exists, rather than two versions (before and after a commit) for behavioral comparison. It provides execution feedback for one program version and lists all covered files and functions. Figure 1 shows the prompt used for *unguided*-Cleverest to generate test inputs for JerryScript.

4.4 Result

Among the 36 bugs across 72 commits, *unguided*-Cleverest can trigger only 4 bugs. It also detects 4 behavior differences. Additionally, it reaches only 10 bug-introducing commits and 6 bug-fixing commits. It performs significantly worse than Cleverest, which triggers 20 bugs and reaches at least 56 changed code locations across 72 commits. This result suggests that incorporating commit messages into LLM-based regression test generation is highly effective, leading to improved bug detection and broader code coverage.

5 Results on 50 Recent Non-Bug Commits of Libxml2

In the main paper, we focus on generating regression tests for commits that introduce or fix bugs, as existing benchmarks and tools provide meaningful comparisons. However, we also argue that Cleverest can be valuable in a CI/CD setting, where tests are generated for commits without prior knowledge of whether they introduce bugs.

To evaluate this, we selected 50 recent non-bug commits from Libxml2 v2.13.3 (released in July 2024) and ran Cleverest on them in 'Gen. cmd' mode, where the LLM must also determine the appropriate execution command. The commits were automatically selected by traversing commits before Libxml2 v2.13.3 in reverse chronological order, retaining those with source code changes while excluding commits that contained keywords indicative of non-functional modifications, such as documentation updates, example additions, and build system changes. We chose Libxml2 because it is under active development and has a large number of well-structured commit messages, ensuring a robust automatic selection process. The full list of 50 commits is available at https://anonymous.4open.science/r/cleverest-7767/.

Summary of Results. Cleverest generated five test cases per commit for all 50 commits in under 40 minutes, averaging 47 seconds per commit. Among these, the generated tests:

- Triggered output behavior differences for six commits.
- Reached changed code for 25 commits in the remaining cases.

In total, Cleverest successfully reached the changed code in 31 out of 50 cases (62%).

Without human intervention, Cleverest effectively generates regression tests that exercise the changed code in non-bug commits, making it a useful tool in CI/CD workflows for ensuring software quality. In practice, developers can:

+ Use Cleverest-generated test cases as seeds for fuzzing specific software versions, potentially uncovering latent bugs, or
+ Manually refine test cases that fail to reach the changed code, constructing high-quality regression tests.

6 Commit Length Statistics

We argue that commit quality plays a crucial role in Cleverest's ability to generate effective regression tests. Commits with detailed messages and atomic code changes help LLMs understand the context better, leading to higher-quality test case generation. In contrast, commits with vague messages or scattered code changes make it difficult for the LLM to extract key information for testing relevant features.

To evaluate this, we analyze the commit message size and code change size across all 72 commits in our benchmark.

- Commit message size is measured by the number of words and characters in the commit message.
- Code change size is measured by the number of lines and words in the code diff.

Table 2. Commit Message and Code Diff Statistics

Metric	Bug-Introducing Commits	Bug-Fixing Commits
Commit Message Size		
Average (words)	44	29
Average (characters)	304	201
Min (words, characters)	(4, 41)	(2, 10)
Max (words, characters)	(208, 1428)	(152, 979)
Code Diff Size		
Average (lines)	494	51
Average (characters)	18.9k	1.9k
Min (lines, characters)	(26, 882)	(12, 411)
Max (lines, characters)	(3291, 140k)	(280, 10.2k)

Table 2 shows the statistics for bug-introducing and bug-fixing commits.

Impact on Cleverest's Performance. By analyzing cases where Cleverest underperforms (i.e., failing to reach commit-changed code), we observe that commit quality affects its effectiveness. We have already discussed in the main text that Cleverest is negatively impacted when:

- Commit messages are too small or vague (e.g., bug-fixing commits for Z3 #6659, #6914, and #7246).
- Code diffs are too large and scattered (e.g., bug-introducing commits for Poppler #1303 and #1381).

Beyond these cases, we do not find a strong correlation between commit message length and the overall effectiveness of Cleverest.

References

[1] Chun Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2023. Fuzz4ALL: Universal Fuzzing with Large Language Models. *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)* (2023), 1547–1559. https://api.semanticscholar.org/CorpusID:260735598